Séries temporelles

Jour 3 · Application au deep learning

ING3 option IA · CY Tech



Le cas d'étude de la séance

Pourquoi : les notions du jour s'appliquent à ce cas, le matin comme au TD ; l'étape 9 les reprend sur une pompe.

Un transformateur électrique, relevé toutes les heures pendant deux ans

- **Données** : 17 420 relevés horaires, du 01/07/2016 au 26/06/2018 (jeu ETTh1)
- **Sept mesures** : six charges électriques et la **température d'huile**
- **Pourquoi l'huile** : trop chaude, elle signale un transformateur en surcharge

Prévoir les 24 prochaines heures de température

Détecter un comportement anormal des 7 mesures

Les trois questions de la séance

1 — **Comment préparer une série sans tricher ?** — partie 1

découper dans l'ordre du temps, normaliser, couper en fenêtres

2 — **Quel modèle pour prévoir ?** — parties 2 à 6

trois baselines face à trois réseaux : LSTM, TCN, Transformer

3 — **Comment repérer une anomalie, et quand alerter ?** — parties 7 et 8

un autoencodeur face à deux méthodes classiques, puis un seuil

Source : jeu ETT (Electricity Transformer Temperature), fichier ETTh1 : Zhou et al., Informer, AAAI 2021.

Plan de la matinée

Pourquoi : le cours suit l'ordre du TD : chaque partie prépare une étape de l'après-midi.

9h30 · avant la pause

Partie 1 — 1
Préparer une série
découper, normaliser, fenêtres
TD · étape 2

Partie 2 — 2
Les baselines
naïve, saisonnière, linéaire
TD · étape 3

Partie 3 — 2
Le LSTM
lire la série pas à pas
TD · étape 4

Partie 4 — 2
Le TCN
convolutions causales et dilatées
TD · étape 4

après la pause

Partie 5 — 2
PatchTST
l'attention entre morceaux
TD · étape 4

Partie 6 — 2
Comparer des prévisions
normaliser chaque fenêtre
TD · étape 4

Partie 7 — 3
L'autoencodeur
détecter une anomalie
TD · étape 5

Partie 8 — 3
Le seuil d'alerte
précision, rappel, coût
TD · étapes 5 et 6

Supposé connu : Python et PyTorch ; la convolution et l'attention (Jour 1) ; validation, test et baseline (Jour 1) ; précision, rappel et AP (Jour 2).



Partie 1

Préparer une série temporelle

Deux règles pour ne pas tricher sans le vouloir, et la forme des données de tous les modèles

Préparer une série temporelle : définition

Définition

Une série temporelle est une suite de mesures prises à intervalles réguliers, ici une par heure. La préparer, c'est la découper dans l'ordre du temps, la normaliser sans regarder l'avenir, puis la couper en fenêtres : un morceau de passé, et la suite à prévoir.

Le problème que cela résout

Mélanger les instants, ou normaliser avec des valeurs futures, donne un excellent score au TD et un modèle inutilisable en service : il a vu l'avenir pendant l'entraînement.

Ce qui entre, ce qui sort

Entrée : un tableau de 17 420 heures et 7 mesures. Sortie : des paires (96 heures de passé, 24 heures à prévoir) pour l'entraînement, la validation et le test.

Ce que l'on va voir, dans l'ordre

2. Une série temporelle
3. Tendance, saisonnalité, bruit
4. Règle 1 : découper dans l'ordre du temps
5. Règle 2 : normaliser avec l'entraînement seulement
6. Les fenêtres glissantes
7. Fenêtres en code
8. À retenir

Dans le TD : Étape 2 : TODO 1 (découper et normaliser) et TODO 2 (fenêtres).

Une série temporelle

Pourquoi : savoir ce que contient le fichier du TD, et les deux questions qu'on lui pose.

Une **série temporelle** : des mesures relevées à intervalles réguliers. Ici, **une ligne par heure** pendant deux ans. L'ordre des lignes compte : c'est le temps.

heure	6 charges électriques	température d'huile
01/07/2016 à 0 h	6 valeurs	30,5 °C
01/07/2016 à 1 h	6 valeurs	27,8 °C
01/07/2016 à 2 h	6 valeurs	27,8 °C
… 17 420 lignes	…	…

Prévoir on n'utilise que la **température d'huile** : ses 96 dernières heures, pour prévoir les 24 suivantes

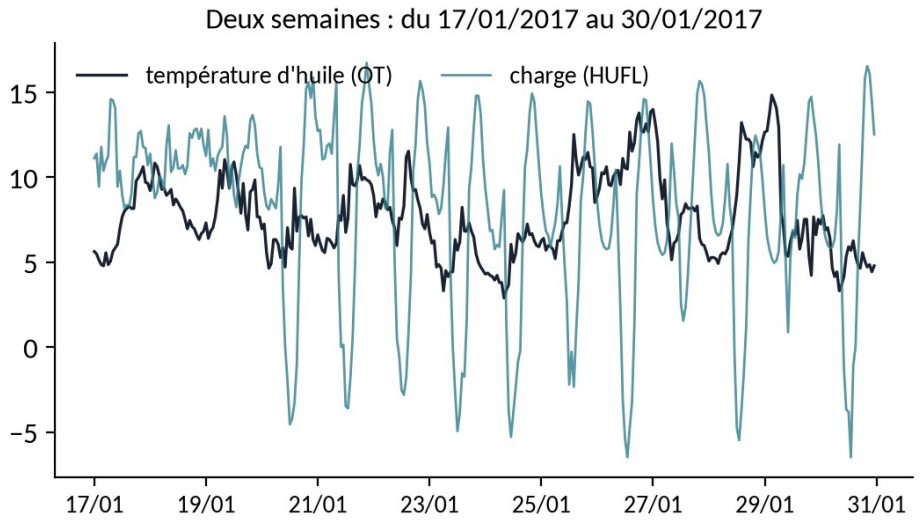
Détecter on surveille les **7 mesures ensemble** : un comportement anormal peut toucher n'importe laquelle

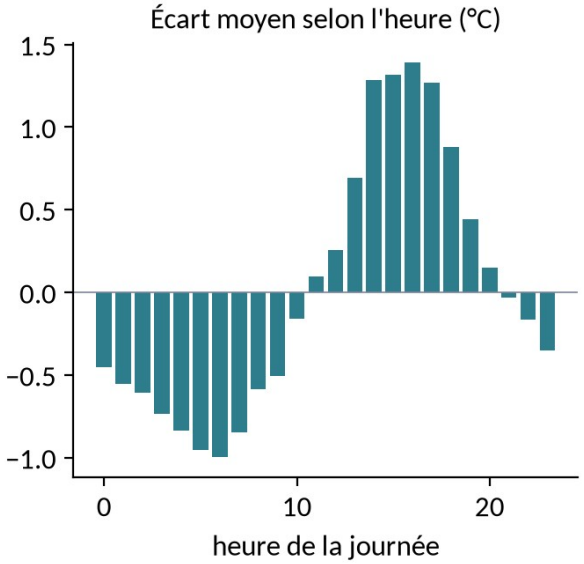
Source : fichier ETTh1 du jeu ETT, premières lignes (valeurs arrondies).

Tendance, saisonnalité, bruit

Pourquoi : trois mots pour décrire une série avant de la modéliser.

Pour décrire une série, trois mots : la **tendance** (le niveau sur des mois), la **saisonnalité** (un motif qui revient) et le **bruit** (le reste).





Tendance

le niveau monte ou descend sur des mois

Saisonnalité

ici, la journée : en moyenne 2,4 °C entre l'heure la plus chaude et la plus froide

Bruit

le reste, imprévisible

La saisonnalité donne une prévision gratuite : recopier la même heure, la veille. C'est l'une des baselines de la partie 2.

Source : ETTh1, calculé pour le cours ; vocabulaire : Hyndman et Athanasopoulos, Forecasting: Principles and Practice, section 2.3.

Règle 1 : découper dans l'ordre du temps

Pourquoi : c'est la règle qui rend une mesure de prévision honnête.

En service, un modèle prévoit **toujours l'avenir à partir du passé**. On le teste donc de la même façon : on s'entraîne sur le début, on teste sur la fin.

Le découpage du TD



Les derniers mois ne servent pas : 12 + 4 + 4 mois, c'est le découpage fixé par les auteurs du jeu et repris par les articles. Le garder permet de comparer ses résultats aux leurs.

À ne pas faire : tirer les heures au hasard



Deux heures voisines se ressemblent : si l'heure 15 sert à l'entraînement et l'heure 16 au test, la « prévision » devient une copie. Le score est excellent, et faux.

Source : Hyndman et Athanasopoulos, Forecasting: Principles and Practice, section 5.10 ; découpage 12, 4 et 4 mois habituel pour ETTh1.

Règle 2 : normaliser avec l'entraînement seulement

Pourquoi : la deuxième fuite d'information à éviter, celle du TODO 1.

On met chaque mesure à la même échelle : **(valeur − moyenne) / écart-type**. La moyenne et l'écart-type viennent de **l'entraînement seulement**, puis servent tels quels partout.

Pour la température d'huile

17,1 °C

moyenne sur les 12 mois d'entraînement

9,2 °C

écart-type sur les mêmes 12 mois

- 26,3 °C devient (26,3 − 17,1) / 9,2 = **1**
- 7,9 °C devient **−1**

À ne pas faire

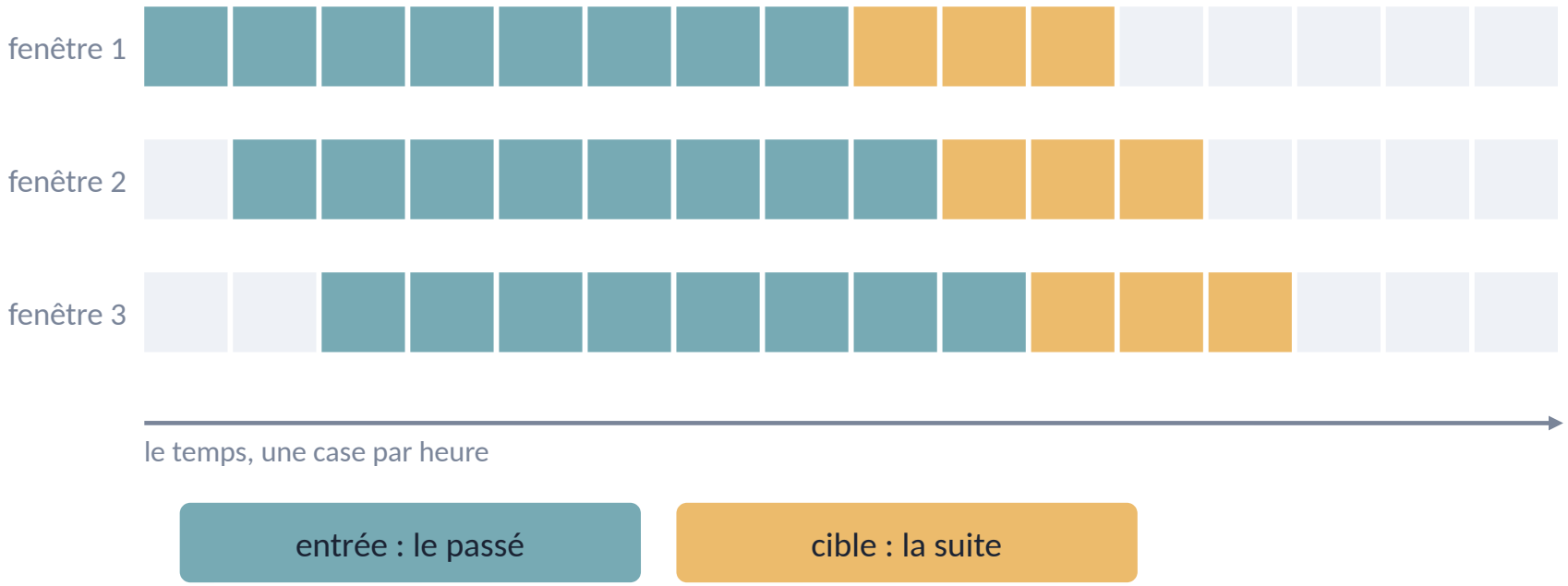
- Calculer la moyenne sur **tout le fichier**, test compris
- Le modèle apprend alors avec une information sur l'avenir : une **fuite**
- Au moment de l'entraînement, en service, le test n'existe pas encore

Au TD, une erreur de 1 en unité normalisée vaut **9,2 °C** : c'est ainsi que le notebook affiche les erreurs en degrés.

Les fenêtres glissantes

Pourquoi : c'est la forme des données que reçoivent tous les modèles du TD.

Le modèle reçoit une **fenêtre de passé** et doit prévoir **la suite**. On fait glisser la fenêtre d'une heure : chaque heure donne un nouvel exemple.



Au TD

- **96 heures** d'entrée, 4 jours
- **24 heures** à prévoir
- **8 521** fenêtres d'entraînement
- **2 761** de validation, autant de test

Nombre de fenêtres, pour les curieux : longueur − entrée − horizon + 1. Entraînement : 8 640 − 96 − 24 + 1 = 8 521.

Fenêtres en code

Pourquoi : voir les fenêtres sur un exemple que l'on vérifie de tête.

```
import numpy as np
from numpy.lib.stride_tricks import sliding_window_view

serie = np.arange(10.0)                  # 10 relevés : 0, 1, ..., 9
entree, horizon = 4, 2                   # 4 heures connues, 2 à prévoir

tranches = sliding_window_view(serie, entree + horizon)   # tranches de 6
X, Y = tranches[:, :entree], tranches[:, entree:]
print(X.shape, Y.shape)        # (5, 4) (5, 2) : 5 fenêtres
print(X[0], "->", Y[0])        # [0. 1. 2. 3.] -> [4. 5.]
print(X[1], "->", Y[1])        # [1. 2. 3. 4.] -> [5. 6.]
print(X[-1], "->", Y[-1])      # [4. 5. 6. 7.] -> [8. 9.]
```

- Une série de 10 valeurs, **4** heures d'entrée, **2** à prévoir
- **sliding_window_view** donne toutes les tranches de 6 valeurs consécutives
- Les 4 premières forment X, les 2 dernières Y : **10 − 4 − 2 + 1 = 5** fenêtres
- Au TODO 2, vous écrivez la même chose avec une boucle

Tous les extraits de code du cours ont été exécutés avec les versions de uv.lock (PyTorch 2.14.1, NumPy) : les sorties écrites en commentaire sont celles obtenues.

Source : Documentation NumPy : numpy.lib.stride_tricks.sliding_window_view.

Préparer une série temporelle : à retenir

1. On découpe **dans l'ordre du temps** : l'entraînement, puis la validation, puis le test. Jamais de tirage au hasard.

2. On normalise avec la **moyenne et l'écart-type de l'entraînement**, appliqués tels quels à la validation et au test.

3. Une **fenêtre** : 96 heures de passé en entrée, les 24 heures suivantes comme cible. On la fait glisser d'une heure à la fois.

Pour approfondir : Forecasting: Principles and Practice, section 2.3 : Time series patterns ; d2l, section 9.1 : Working with Sequences

Partie 2

Les baselines de prévision

Savoir ce que vaut une prévision simple avant de payer pour un réseau

Les baselines de prévision : définition

Définition

Une baseline est une prévision simple qui sert de point de comparaison. Deux sont sans apprentissage : répéter la dernière valeur (naïve), ou recopier la veille à la même heure (saisonnière). Une troisième apprend, mais sans réseau : la régression linéaire.

Le problème que cela résout

Sans point de comparaison, une erreur de 1 °C ne dit rien. Un réseau qui ne bat pas « demain comme maintenant » ne vaut pas son coût.

Ce qui entre, ce qui sort

Entrée : une fenêtre de 96 heures. Sortie : les 24 heures prévues, et leur erreur moyenne en degrés (la MAE).

Ce que l'on va voir, dans l'ordre

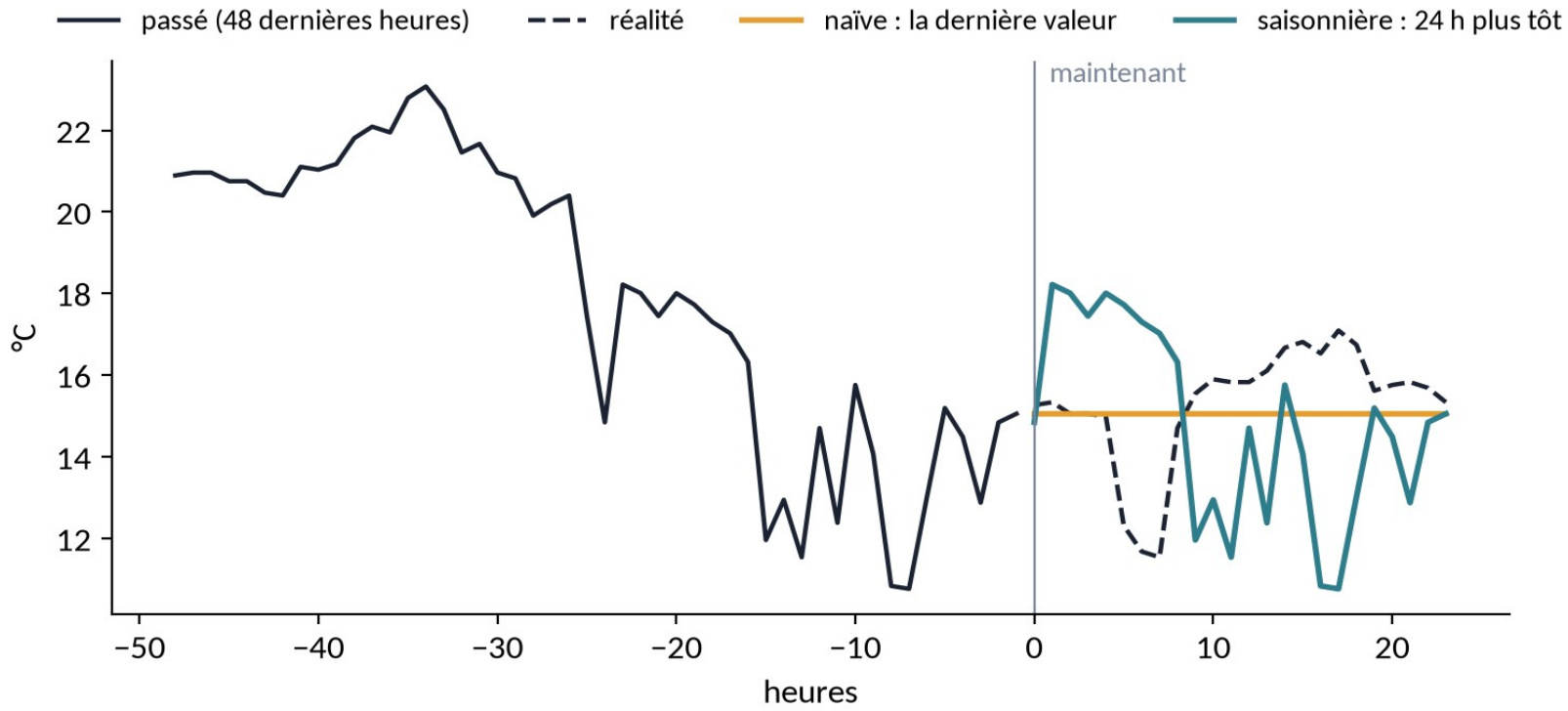
2. Deux prévisions sans apprentissage
3. Une baseline apprise : la régression linéaire
4. Mesurer l'erreur : la MAE
5. Baselines en code
6. À retenir

Dans le TD : Étape 3 : TODO 3 (naïve et saisonnière) ; la régression linéaire est fournie.

Deux prévisions sans apprentissage

Pourquoi : les deux points de comparaison les plus simples, ceux du TODO 3.

Naïve : demain ressemblera à maintenant, on répète la dernière valeur. **Saisonnière** : demain à 15 h comme aujourd'hui à 15 h, on recopie la veille.



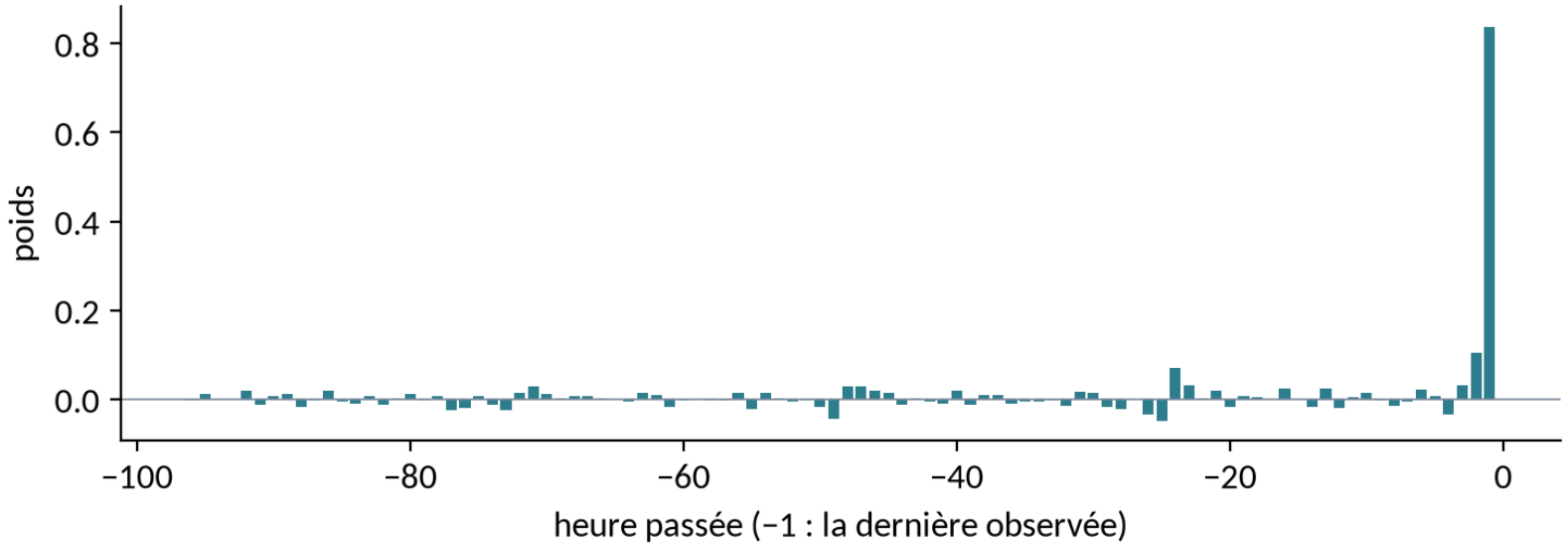
- **Aucun apprentissage** : rien à entraîner, aucun paramètre
- Une seule fenêtre **ne suffit pas** pour conclure : le TD les mesure sur toutes les fenêtres de test

Source : ETTh1, une fenêtre de validation, calculé pour le cours ; méthodes : Forecasting: Principles and Practice, section 5.2.

Une baseline apprise : la régression linéaire

Pourquoi : une baseline plus exigeante : elle apprend, mais sans réseau.

Chaque heure prévue est une **somme pondérée des 96 heures passées**. Les poids sont appris sur l'entraînement, sans réseau de neurones.



Les 96 poids appris pour la première heure prévue.

2 328

paramètres : 96 poids et une constante, pour chacune des 24 heures

0,84

le poids de la dernière heure : la régression retrouve presque la prévision naïve

Ridge : une pénalité empêche les poids de devenir trop grands. Ajustement en une fraction de seconde (0,01 s mesurée).

Source : ETTh1, Ridge(alpha=10) de scikit-learn ajustée sur les fenêtres d'entraînement du TD, calculé pour le cours.

Mesurer l'erreur : la MAE

Pourquoi : la mesure de toutes les prévisions du TD, lisible en degrés.

MAE (Mean Absolute Error) : la moyenne des écarts entre prévision et réalité, sans leur signe. Elle se lit **en degrés**.

	heure 1	heure 2	heure 3
réalité	20 °C	22 °C	21 °C
prévision	21 °C	21 °C	21 °C
écart, sans signe	1	1	0

0,67 °C

MAE = (1 + 1 + 0) / 3

Plus la MAE est basse, mieux c'est. Au TD, elle est moyennée sur les 24 heures de chaque fenêtre et sur toutes les fenêtres de test.

Source : Hyndman et Athanasopoulos, Forecasting: Principles and Practice, section 5.8.

Baselines en code

Pourquoi : les deux baselines et la MAE, sur un exemple que l'on vérifie de tête.

```python
import numpy as np

# un cycle de 4 heures qui monte d'un degré à chaque cycle
serie = np.array([10., 14, 12, 8, 11, 15, 13, 9, 12, 16, 14, 10])
passe, reel = serie[:8], serie[8:]      # 8 heures connues, 4 à prévoir

naive = np.repeat(passe[-1], 4)         # la dernière valeur, répétée
saisonniere = passe[-4:]                # un cycle plus tôt

def mae(prevision):
    return np.abs(prevision - reel).mean()

print(naive, mae(naive))                # [9. 9. 9. 9.] 4.0
print(saisonniere, mae(saisonniere))    # [11. 15. 13.  9.] 1.0
```

- Une série qui suit un **cycle de 4 heures**, et monte d'un degré à chaque cycle
- **Naïve** : la dernière valeur, 9, répétée
- **Saisonnière** : les 4 dernières valeurs, recopiées
- Ici, la saisonnière gagne : **1,0** contre **4,0**

Les baselines de prévision : à retenir

1. **Naïve** : la dernière valeur, répétée. **Saisonnière** : la valeur de la même heure, 24 heures plus tôt. Aucun paramètre à apprendre.

2. La **régression linéaire** (Ridge) : chaque heure prévue est une somme pondérée des 96 heures passées ; 2 328 paramètres, apprise en une fraction de seconde.

3. La **MAE** : l'écart moyen entre prévision et réalité, en degrés. Plus elle est basse, mieux c'est.

Pour approfondir : Forecasting: Principles and Practice, section 5.2 : Some simple forecasting methods ; section 5.8 : Evaluating point forecast accuracy



Partie 3

Le LSTM, un réseau récurrent

Premier réseau : il lit la série dans l'ordre, une heure après l'autre

Le LSTM : définition

Définition

Un réseau récurrent lit la série une valeur à la fois et garde un résumé de ce qu'il a lu : son état caché, sa mémoire. Le LSTM (Long Short-Term Memory) est un réseau récurrent dont des portes décident, à chaque pas, quoi oublier, quoi retenir et quoi transmettre.

Le problème que cela résout

Un réseau dense apprend un poids distinct pour chaque heure de la fenêtre. Le réseau récurrent suit le temps avec une seule cellule, réutilisée à chaque heure, et sa mémoire résume tout ce qu'il a lu.

Ce qui entre, ce qui sort

Entrée : une fenêtre de 96 heures, lue heure par heure. Sortie : l'état caché final (64 nombres), qu'une couche linéaire transforme en 24 heures prévues.

Ce que l'on va voir, dans l'ordre

2. Lire la série pas à pas
3. Les portes du LSTM
4. Les limites des réseaux récurrents
5. Un LSTM de prévision en PyTorch
6. À retenir

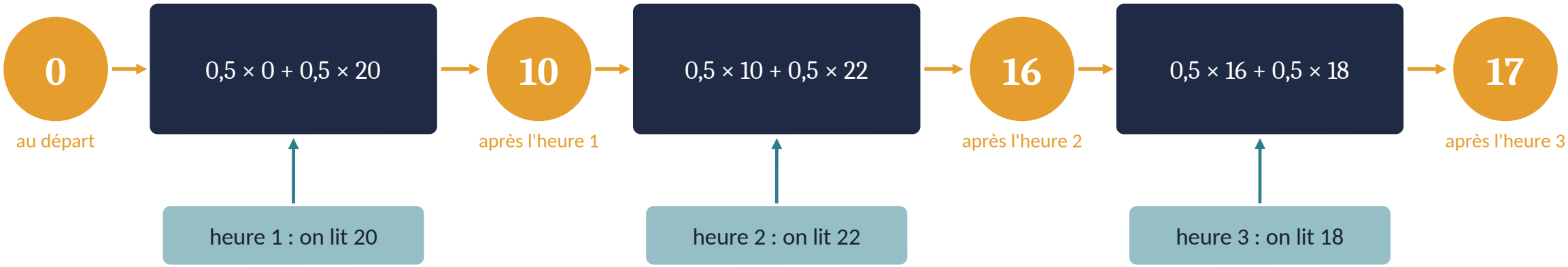
Dans le TD : Étape 4 : le LSTM est fourni, prêt à entraîner.

Lire la série pas à pas

Pourquoi : comprendre comment un réseau suit l'ordre du temps.

Le réseau lit la fenêtre **une heure à la fois** et tient à jour une **mémoire** (l'état caché) : à chaque heure, la nouvelle mémoire **mélange l'ancienne mémoire et la valeur lue**.

règle jouet : nouvelle mémoire = 0,5 × ancienne mémoire + 0,5 × valeur lue



Et ainsi de suite jusqu'à l'heure 96. Après 3 heures : 17 = 0,5 × 18 + 0,25 × 22 + 0,125 × 20 ; chaque heure compte, les plus anciennes de moins en moins.

Un vrai réseau récurrent : coefficients **appris**, mémoire de **64 nombres**, et la **même règle** pour les 96 heures. À la fin, la mémoire résume la fenêtre ; une couche linéaire en tire les 24 heures prévues.

Source : d2l, chapitre 9 (Recurrent Neural Networks).

Les portes du LSTM

Pourquoi : savoir ce que fait la cellule du LSTM, sans entrer dans ses équations.

Le LSTM remplace les coefficients fixes par des **portes** : des nombres entre 0 et 1 que le réseau **recalcule à chaque heure**, selon ce qu'il lit. Il décide ainsi, au cas par cas, quoi **oublier** et quoi **retenir**.

nouvelle mémoire = oubli × ancienne mémoire + entrée × nouvelle information

Exemple : la mémoire vaut 16, un pic à 30 arrive.

Ignorer le pic

oubli = 1, entrée = 0

1 × 16 + 0 × 30 = 16

Repartir du pic

oubli = 0, entrée = 1

0 × 16 + 1 × 30 = 30

Mélanger

oubli = 0,5, entrée = 0,5

0,5 × 16 + 0,5 × 30 = 23

Une troisième porte, la **sortie**, dit quelle part de la mémoire est transmise. Le **GRU** fait de même avec deux portes.

Pour les curieux : $c_t = f_t \times c_{t-1} + i_t \times \tilde{c}_t$ et $h_t = o_t \times \tanh(c_t)$, où f, i, o sont les portes (oubli, entrée, sortie), c la mémoire, $\tilde{c}$ la nouvelle information (une petite couche, ici simplifiée en valeur lue), h l'état caché transmis.

Source : Hochreiter et Schmidhuber, Long Short-Term Memory, Neural Computation 1997 ; d2l, section 10.1.

Les limites des réseaux récurrents

Pourquoi : comprendre pourquoi d'autres architectures ont été proposées.

Deux limites, qui ont conduit aux deux familles suivantes.

La mémoire se dilue

Une valeur d'il y a 96 heures a traversé 96 mises à jour avant la sortie. Les portes atténuent le problème sans le supprimer.

Le calcul est séquentiel

L'heure 50 attend le résultat de l'heure 49 : impossible de traiter toutes les heures en même temps.

Partie 4 · le TCN

des convolutions : toutes les heures en même temps

Partie 5 · PatchTST

l'attention : chaque morceau voit tous les autres

Source : d2l, section 11.6.2 : comparaison des CNN, des RNN et de l'attention.

Un LSTM de prévision en PyTorch

Pourquoi : le modèle de l'étape 4, et la forme de chaque tenseur.

```python
import torch
import torch.nn as nn

lstm = nn.LSTM(input_size=1, hidden_size=64, batch_first=True)
tete = nn.Linear(64, 24)

x = torch.randn(32, 96)                    # 32 fenêtres de 96 heures
sorties, (h, c) = lstm(x.unsqueeze(-1))    # entrée (32, 96, 1)
print(sorties.shape)                       # torch.Size([32, 96, 64])
                                           # l'état caché après chaque heure
dernier = sorties[:, -1]                   # le dernier résume la fenêtre
print(dernier.shape)                       # torch.Size([32, 64])
prevision = tete(dernier)
print(prevision.shape)                     # torch.Size([32, 24]) : 24 heures

n = sum(p.numel() for m in (lstm, tete) for p in m.parameters())
print(n)                                   # 18712 paramètres
```

- **nn.LSTM** attend (lot, temps, valeurs) : d'où **unsqueeze(-1)**, une valeur par heure
- **sorties** : l'état caché après chacune des 96 heures
- On garde **le dernier**, puis une couche linéaire donne 24 heures
- **18 712** paramètres : le modèle de l'étape 4

Source : Documentation PyTorch : torch.nn.LSTM.

Le LSTM : à retenir

1. Un réseau récurrent lit la série **pas à pas** et met à jour un **état caché** : le résumé de tout ce qu'il a lu.

2. Le **LSTM** ajoute des **portes** (oublier, retenir, transmettre) qui protègent sa mémoire ; le **GRU** en est une version allégée, à deux portes.

3. Deux limites : l'information ancienne **se dilue** au fil des pas, et le calcul est **séquentiel**, une heure après l'autre.

Pour approfondir : d2l, section 10.1 : Long Short-Term Memory (LSTM) ; C. Olah, Understanding LSTM Networks



Partie 4

Le TCN, des convolutions sur le temps

Deuxième réseau : la convolution du Jour 1, rendue causale et dilatée

Le TCN : définition

Définition

Un TCN (Temporal Convolutional Network) est un réseau qui empile une seule sorte de brique : la convolution 1D, où un petit filtre glisse le long du temps. Chaque convolution est à la fois causale, elle ne lit que le passé, et dilatée, son filtre saute des points pour voir plus loin.

Le problème que cela résout

Le LSTM avance une heure après l'autre. Une convolution traite tous les instants en même temps, et la dilatation lui fait voir 96 heures avec peu de couches.

Ce qui entre, ce qui sort

Entrée : une fenêtre de 96 heures. Sortie : une représentation de chaque instant ; celle du dernier instant donne les 24 heures prévues.

Ce que l'on va voir, dans l'ordre

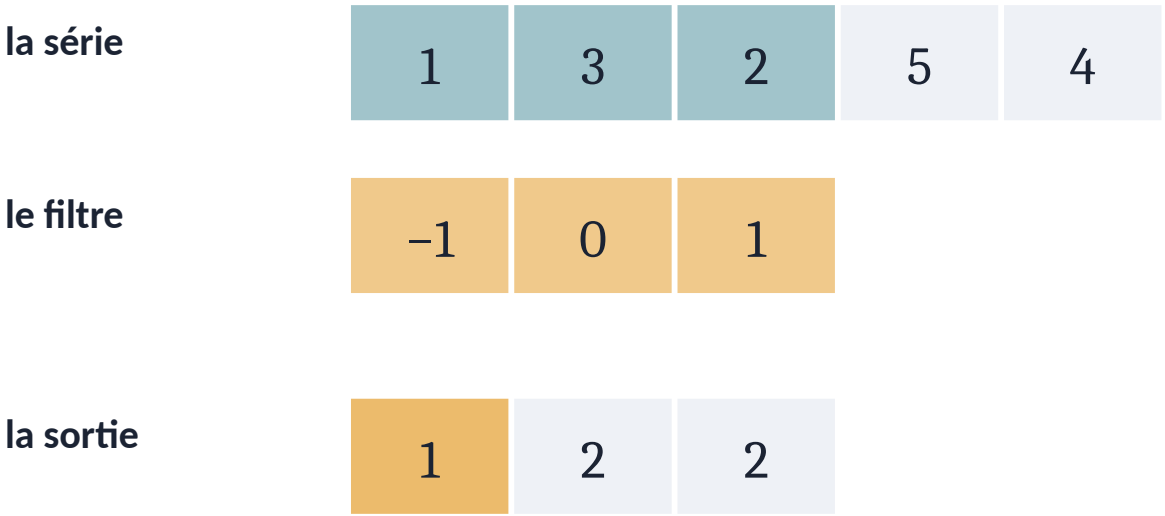
2. La convolution 1D
3. Causale : ne jamais regarder le futur
4. Dilatée : voir loin avec peu de couches
5. Le TCN du TD : cinq blocs
6. Convolution causale en PyTorch
7. À retenir

Dans le TD : Étape 4 : TODO 4 (la convolution causale).

La convolution 1D

Pourquoi : la même brique qu'au Jour 1, sur le temps au lieu de l'image.

La convolution du Jour 1, sur **une seule dimension** : un petit filtre glisse **le long du temps**. À chaque position : on multiplie, puis on additionne.



Première sortie

$$-1 \times 1 + 0 \times 3 + 1 \times 2 = 1$$

Puis le filtre avance d'une case : 5 − 3 = 2, puis 4 − 2 = 2.
Ce filtre répond fort à une **montée**.

Comme en vision, les poids du filtre sont **appris**, pas choisis à la main.

Causale : ne jamais regarder le futur

Pourquoi : une prévision ne doit utiliser que le passé ; c'est le TODO 4.

Pour prévoir, la sortie à l'instant t ne doit lire **que t et le passé**. La recette : ajouter des **zéros à gauche seulement**.

Convolution ordinaire, un zéro de chaque côté



la sortie à t = 5 lit **x6** : le futur

Convolution causale, deux zéros à gauche



la sortie à t = 5 lit x3, x4, **x5** : rien après t

En surligné : les trois valeurs lues par un filtre de taille 3 pour la sortie à l'instant 5. En pointillés : les zéros ajoutés.

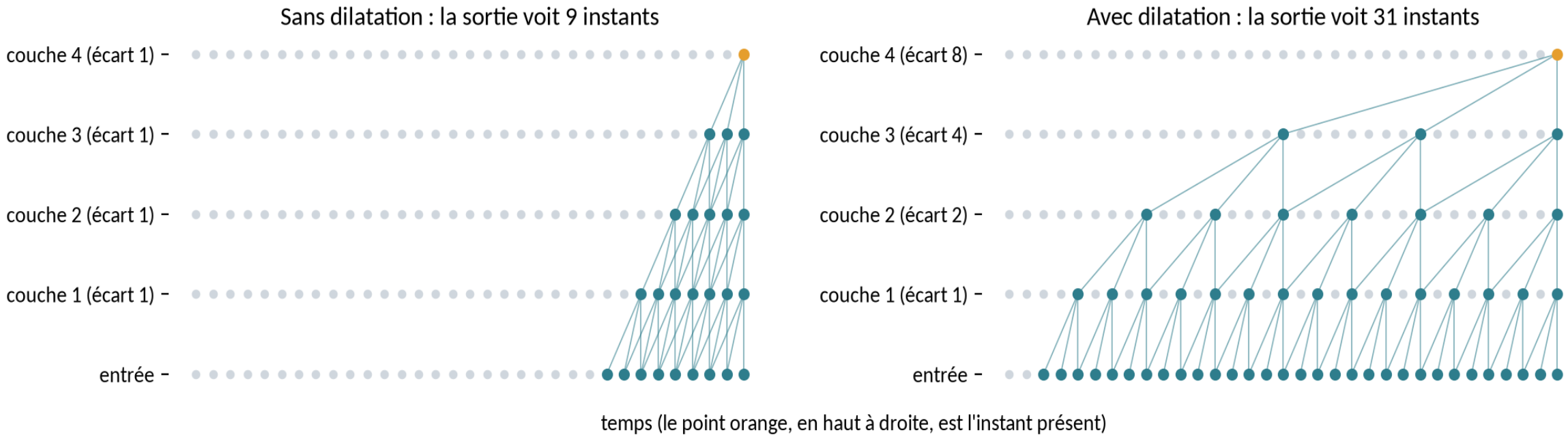
Nombre de zéros à gauche : **(taille du filtre − 1) × dilatation**. La sortie garde la longueur de l'entrée. C'est le TODO 4.

Source : Bai, Kolter et Koltun, An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling, 2018.

Dilatée : voir loin avec peu de couches

Pourquoi : c'est ce qui permet au TCN de voir les 96 heures de la fenêtre.

Dilatée : le filtre saute des points, et l'écart **double à chaque couche**. La longueur de passé que voit la sortie, le **champ réceptif**, grandit très vite.



9 → 31

instants vus avec 4 couches et un filtre de taille 3, sans puis avec dilatation

125 heures

vues par le TCN du TD : 5 blocs, écarts 1, 2, 4, 8, 16 ; la fenêtre en fait 96

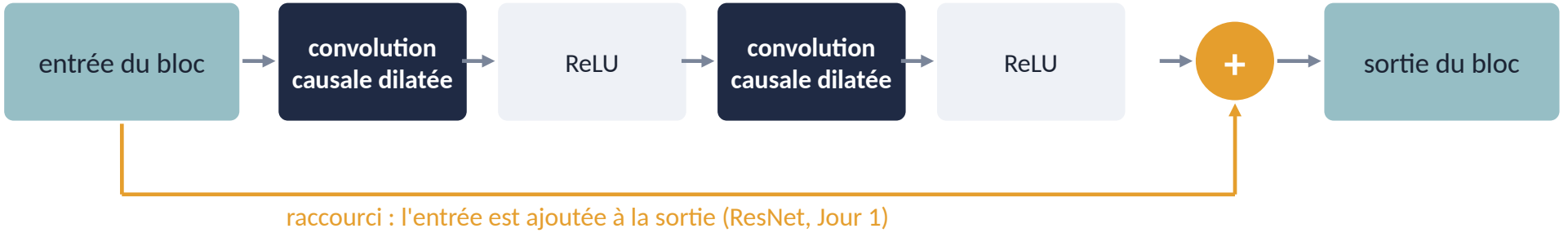
Source : van den Oord et al., WaveNet, 2016 ; Bai, Kolter et Koltun, 2018. Figure calculée pour le cours (noyau 3).

Le TCN du TD : cinq blocs

Pourquoi : voir comment ces convolutions s'assemblent dans le modèle de l'étape 4.

Un TCN n'utilise **qu'une sorte de brique** : la convolution 1D, à la fois **causale et dilatée**. On les regroupe par deux dans des **blocs**, avec un raccourci comme dans ResNet.

Un bloc



Le modèle



32 filtres de taille 3 par convolution · **28 920** paramètres · la sortie voit **125 heures**

Source : Bai, Kolter et Koltun, 2018, figure 1 (bloc résiduel) ; paramètres comptés sur le modèle du TD.

Convolution causale en PyTorch

Pourquoi : vérifier sur des chiffres que la sortie ne dépend pas du futur.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

x = torch.arange(1., 7.).reshape(1, 1, 6)   # (lot, canaux, temps)
conv = nn.Conv1d(1, 1, kernel_size=3, dilation=2, bias=False)
nn.init.ones_(conv.weight)                  # il additionne ses 3 entrées

print(conv(x).shape)                        # torch.Size([1, 1, 2])
marge = (3 - 1) * 2                         # (noyau - 1) x dilatation
y = conv(F.pad(x, (marge, 0)))              # 4 zéros à gauche (passé)
print(y[0, 0].tolist())                     # [1.0, 2.0, 4.0, 6.0, 9.0, 12.0]

x[..., -1] = 100.                           # on change le dernier instant
y = conv(F.pad(x, (marge, 0)))
print(y[0, 0].tolist())                     # [1.0, 2.0, 4.0, 6.0, 9.0, 106.0]
```

- Un filtre de taille 3, **dilatation 2**, poids à 1 : il additionne x(t), x(t − 2) et x(t − 4)
- Sans marge, la série de 6 points tombe à **2**
- **F.pad(x, (4, 0))** : 4 zéros à gauche, la longueur est conservée
- On change le dernier instant : **seule la dernière sortie bouge**

Source : Documentation PyTorch : torch.nn.Conv1d, torch.nn.functional.pad.

Le TCN : à retenir

1. Une **convolution 1D** fait glisser un petit filtre le long du temps : la brique du Jour 1, sur une seule dimension.

2. **Causale** : on ajoute (noyau − 1) × dilatation zéros **à gauche seulement** ; la sortie à l'instant t ne voit que le passé.

3. **Dilatée** : l'écart entre les points lus double d'un étage au suivant (d'un bloc au suivant dans le TD), et le **champ réceptif** grandit très vite. Le TCN du TD voit 125 heures.

Pour approfondir : Bai, Kolter et Koltun, 2018 : l'article TCN (arXiv) ; d2l, section 7.2 : Convolutions for Images



Partie 5

PatchTST, un Transformer à patches

Troisième réseau : l'attention du Jour 1, entre morceaux de série

PatchTST : définition

Définition

PatchTST est un Transformer pour séries temporelles. Il découpe la fenêtre en morceaux de quelques heures, les patches, en fait des tokens, et applique l'attention entre ces morceaux, comme le Vision Transformer du Jour 1 avec les morceaux d'image.

Le problème que cela résout

L'attention entre toutes les heures coûte cher, et une heure isolée porte peu de sens. Un morceau de 16 heures porte une forme : une montée, un creux.

Ce qui entre, ce qui sort

Entrée : une fenêtre de 96 heures, découpée en 11 patches de 16 heures. Sortie : les 24 heures prévues.

Ce que l'on va voir, dans l'ordre

2. Rappel : l'attention
3. Découper la série en patches
4. De patches en prévision
5. Patches en PyTorch
6. À retenir

Dans le TD : Étape 4 : TODO 5 (découper en patches).

Rappel : l'attention

Pourquoi : le mécanisme du Jour 1, et ce qu'il coûte sur une série longue.

Rappel du Jour 1 : chaque élément **regarde tous les autres** et en fait une moyenne pondérée par leur ressemblance. Deux heures éloignées sont reliées **directement**.

9 216

paires si chaque heure est un élément : 96 × 96

121

paires avec 11 morceaux de 16 heures : 11 × 11

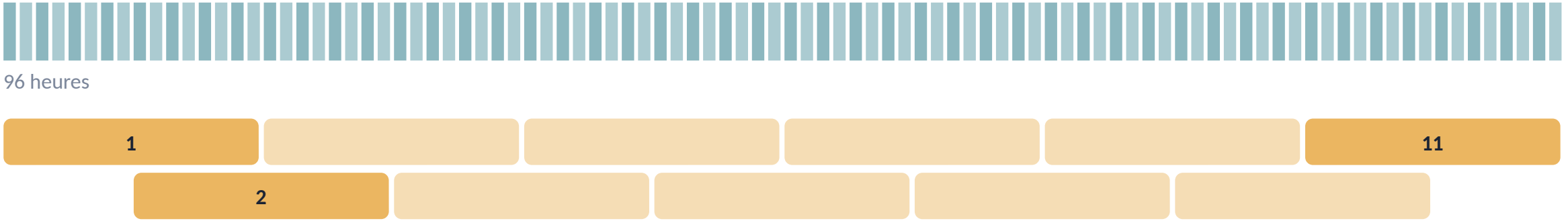
Deux problèmes si chaque heure est un élément : le **nombre de paires** croît comme le carré de la longueur, et **une heure seule** porte peu de sens (18 °C : monte-t-on ou descend-on ?).

Source : Vaswani et al., Attention Is All You Need, NeurIPS 2017 ; d2l, section 11.6.2.

Découper la série en patches

Pourquoi : l'idée de PatchTST, celle du TODO 5.

On découpe la fenêtre en **morceaux de 16 heures**, décalés de 8 : chaque morceau, un **patch**, porte une forme. Comme le ViT découpe une image en carrés.



patch 1

heures 0 à 15

patch 2

heures 8 à 23

patch 11

heures 80 à 95

Nombre de patches : (96 − 16) / 8 + 1 = **11**. Au TODO 5, une ligne : la méthode **unfold** des tenseurs.

Source : Nie et al., A Time Series is Worth 64 Words, ICLR 2023 (patches de 16 pas, décalés de 8).

De patches en prévision

Pourquoi : le chemin complet du modèle de l'étape 4, des patches aux 24 heures.

Chaque patch devient un **token**, reçoit sa **position**, puis l'**encodeur Transformer** fait communiquer les patches. Une couche linéaire donne la prévision.



85 656

paramètres : le plus gros des trois réseaux du TD

Le modèle de l'étape 4 est une version simplifiée de PatchTST : une seule mesure en entrée (la température d'huile), 2 couches d'encodeur.

Source : Nie et al., ICLR 2023, version simplifiée du TD ; paramètres comptés sur le modèle du TD.

Patches en PyTorch

Pourquoi : le découpage en une ligne, et la forme des tokens.

```python
import torch
import torch.nn as nn

x = torch.arange(96.).reshape(1, 96)   # une fenêtre : heures 0 à 95
patches = x.unfold(1, 16, 8)           # 16 heures, décalées de 8
print(patches.shape)                   # torch.Size([1, 11, 16]) : 11 patches
print(patches[0, 0, [0, -1]])          # tensor([ 0., 15.]) : heures 0 à 15
print(patches[0, 1, [0, -1]])          # tensor([ 8., 23.]) : heures 8 à 23

tokens = nn.Linear(16, 64)(patches)    # un patch -> 64 nombres
print(tokens.shape)                    # torch.Size([1, 11, 64]) : 11 tokens
```

- Une fenêtre qui vaut 0 à 95 : chaque valeur est **son heure**
- **unfold(1, 16, 8)** : sur la dimension du temps, des morceaux de 16, décalés de 8
- **11** patches ; le deuxième commence à l'heure 8
- **nn.Linear(16, 64)** fait de chaque patch un token

Source : Documentation PyTorch : torch.Tensor.unfold.

PatchTST : à retenir

1. L'**attention** : chaque élément regarde tous les autres ; le nombre de paires croît comme le **carré** du nombre d'éléments.

2. PatchTST découpe 96 heures en **11 patches** de 16 heures, décalés de 8 : 121 paires au lieu de 9 216.

3. Chaque patch devient un vecteur, reçoit une **position**, puis traverse l'encodeur Transformer ; une couche linéaire en tire les 24 heures prévues.

Pour approfondir : Nie et al., 2023 : l'article PatchTST (arXiv) ; d2l, section 11.8 : Transformers for Vision



Partie 6

Comparer des prévisions

Un piège propre aux séries, et les règles d'une comparaison équitable

Comparer des prévisions : définition

Définition

Comparer des prévisions, c'est entraîner tous les modèles sur les mêmes fenêtres, garder pour chaque réseau la meilleure époque sur la validation, puis mesurer leur MAE une seule fois sur le test, à côté des baselines.

Le problème que cela résout

Le niveau de la température change au fil des mois. Un réseau entraîné autour de 17 °C reçoit en test des valeurs autour de 5 °C, qu'il n'a presque jamais vues.

Ce qui entre, ce qui sort

Entrée : trois baselines et trois réseaux. Sortie : un tableau avec, pour chacun, la MAE de test, le temps d'entraînement et le nombre de paramètres.

Ce que l'on va voir, dans l'ordre

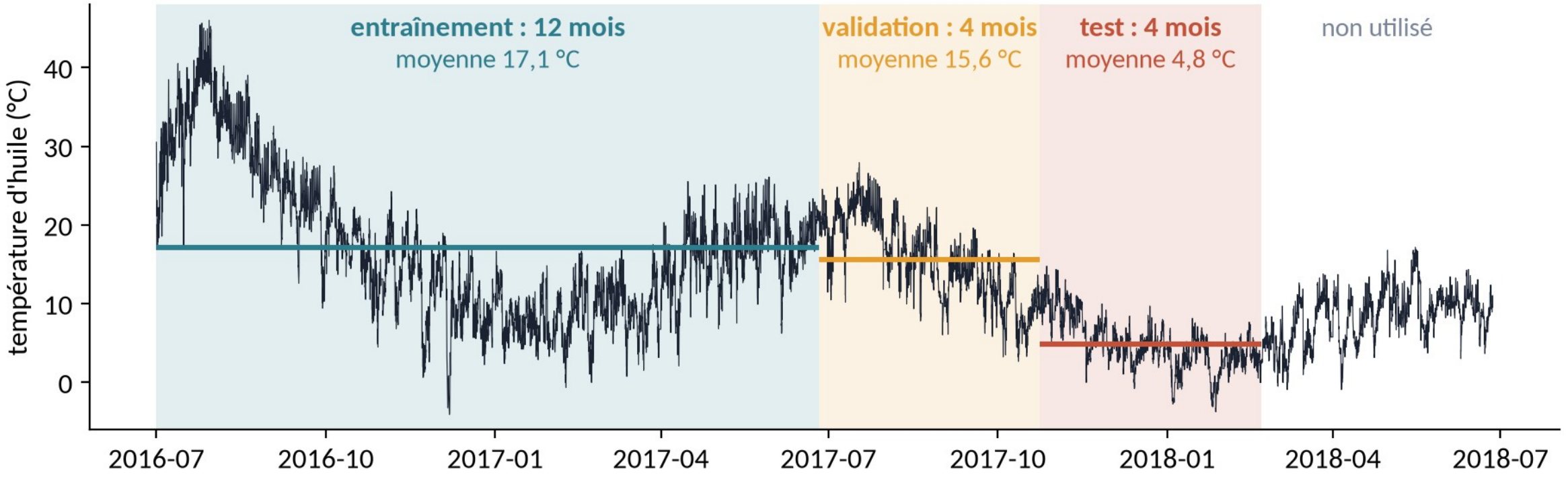
2. Le niveau de la série dérive
3. Normaliser chaque fenêtre
4. Une comparaison équitable
5. À retenir

Dans le TD : Étape 4 : TODO 6 (normaliser chaque fenêtre), puis deux entraînements, sans et avec.

Le niveau de la série dérive

Pourquoi : le piège principal de ce jeu de données.

La période de test est **beaucoup plus froide** que la période d'entraînement : le niveau de la série **dérive**.



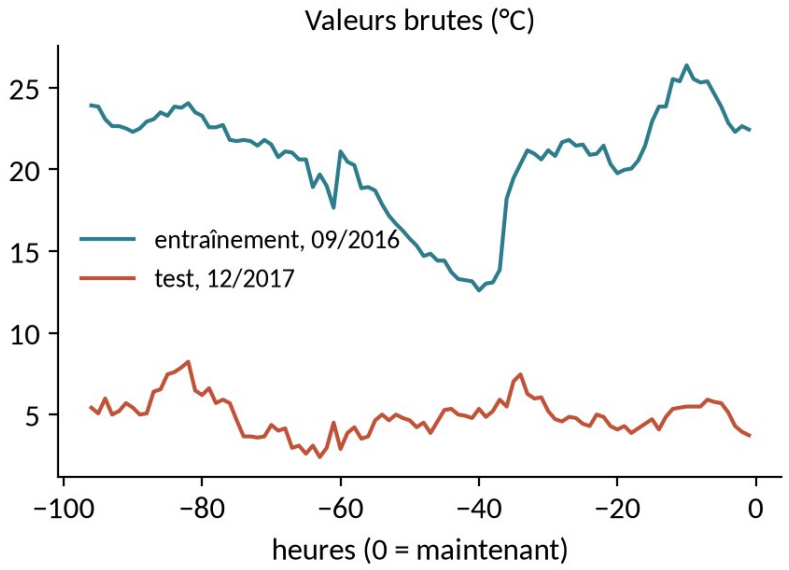
Moyenne de la température d'huile : **17,1 °C** à l'entraînement, **4,8 °C** au test. En test, le réseau reçoit des niveaux qu'il a rarement vus.

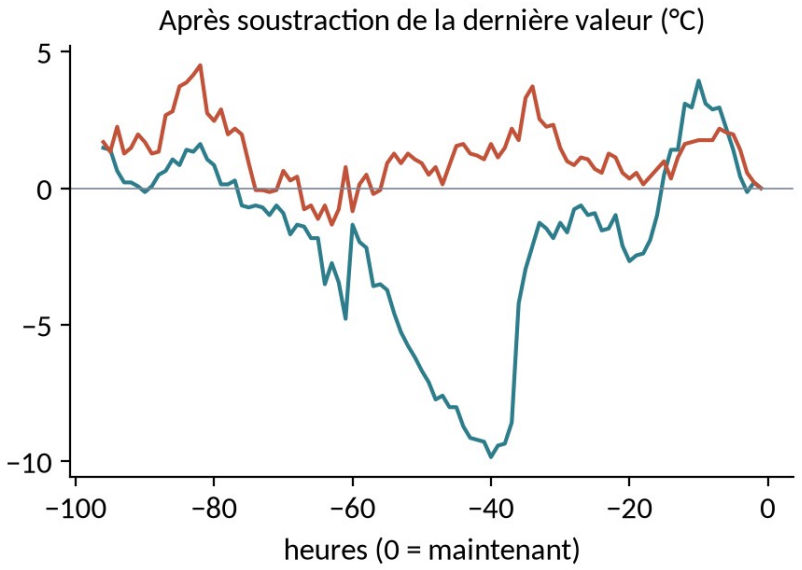
Source : ETTh1, découpage du TD, calculé pour le cours.

Normaliser chaque fenêtre

Pourquoi : une astuce de préparation, celle du TODO 6.

On retire à chaque fenêtre **sa dernière valeur** : le réseau ne voit plus le niveau, seulement **la forme**. Il prédit des écarts, puis on rajoute cette valeur.





1. dernier = la dernière valeur de la fenêtre
2. le réseau reçoit **fenêtre − dernier**
3. prévision = **sortie du réseau + dernier**

Cas particulier : un réseau qui prédit toujours un écart de 0 donne exactement la **prévision naïve**. C'est le TODO 6.

Source : ETTh1, calculé pour le cours ; même idée que le modèle NLinear de Zeng et al., AAAI 2023.

Une comparaison équitable

Pourquoi : les règles qui rendent le tableau final de l'étape 4 lisible.

Mêmes fenêtres, même nombre d'époques, **meilleure époque choisie sur la validation**, test mesuré **une seule fois**. On lance deux fois : sans, puis avec la normalisation par fenêtre.

Modèle	Famille	Paramètres	MAE de test (°C)
Naïve	classique	0	à mesurer au TD
Saisonnière	classique	0	à mesurer au TD
Régression linéaire (Ridge)	classique	2 328	à mesurer au TD
LSTM	deep learning	18 712	à mesurer au TD
TCN	deep learning	28 920	à mesurer au TD
PatchTST	deep learning	85 656	à mesurer au TD

Le tableau du TD ajoute le temps d'entraînement, mesuré sur votre machine.

Source : paramètres comptés sur les modèles du TD (6 époques, perte L1, meilleure époque choisie sur la validation).

Comparer des prévisions : à retenir

1. Le niveau d'une série **dérive** : la période de test peut ne ressembler à aucune période d'entraînement.

2. **Normaliser chaque fenêtre** : retirer sa dernière valeur, prédire des écarts, puis la rajouter. Le réseau apprend la **forme**, et non plus le niveau.

3. Mêmes fenêtres, mêmes époques, meilleure époque choisie **sur la validation**, test mesuré **une fois** ; on compare la MAE, le temps et la taille.

Pour approfondir : Forecasting: Principles and Practice, section 5.10 : Time series cross-validation



Partie 7

Détecter une anomalie avec un autoencodeur

Deuxième tâche : se passe-t-il quelque chose d'anormal ?

L'autoencodeur : définition

Définition

Un autoencodeur est un réseau qui compresse son entrée en un petit code, puis la reconstruit. Entraîné sur des données normales seulement, il reconstruit bien ce qui est normal, et moins bien le reste : l'erreur de reconstruction sert de score d'anomalie.

Le problème que cela résout

Les pannes sont rares : on n'a presque jamais assez d'exemples annotés pour entraîner un classifieur. On apprend donc ce qui est normal, et l'on signale ce qui s'en écarte.

Ce qui entre, ce qui sort

Entrée : une fenêtre de 48 heures sur les 7 mesures. Sortie : un score, l'erreur de reconstruction ; plus il est haut, plus la fenêtre est suspecte.

Ce que l'on va voir, dans l'ordre

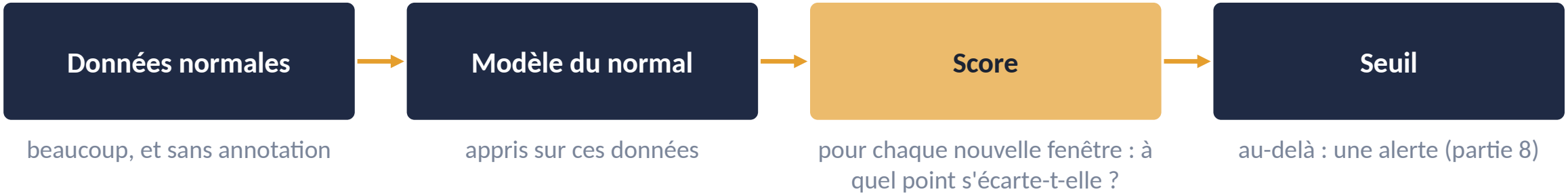
2. Apprendre ce qui est normal
3. Quatre anomalies injectées
4. L'autoencodeur : compresser puis reconstruire
5. L'erreur de reconstruction comme score
6. Les détecteurs classiques : ACP et Isolation Forest
7. L'autoencodeur en PyTorch
8. À retenir

Dans le TD : Étape 5 : TODO 7 (le score), face à l'ACP et à l'Isolation Forest.

Apprendre ce qui est normal

Pourquoi : détecter des pannes sans exemples de pannes.

Les pannes sont **rares** : trop peu d'exemples pour un classifieur. On apprend donc **ce qui est normal**, et l'on signale ce qui s'en écarte.

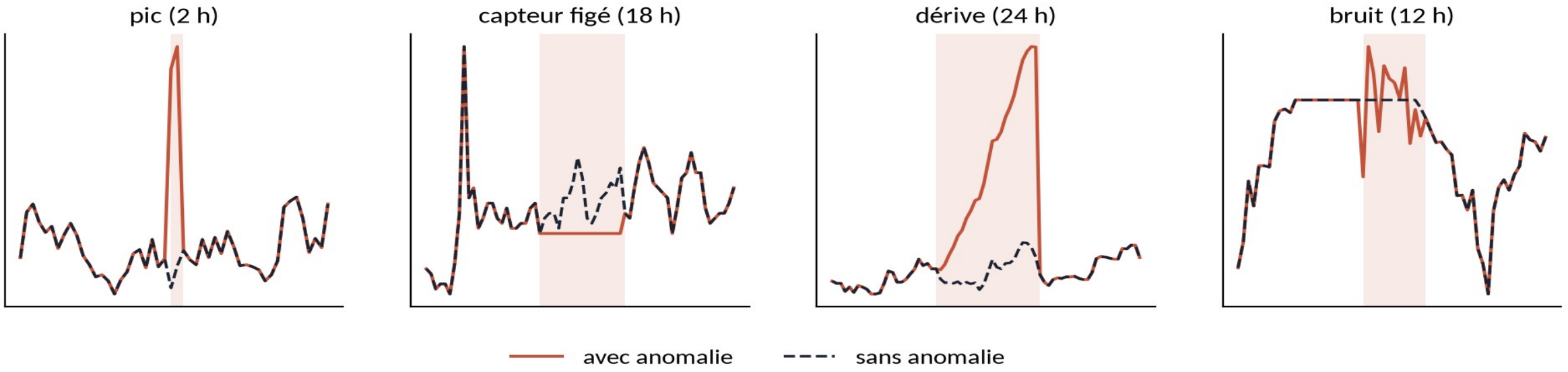


Une anomalie, c'est **ce que le modèle du normal n'explique pas**. Pas besoin d'avoir vu la panne avant.

Quatre anomalies injectées

Pourquoi : savoir ce que les détecteurs doivent trouver au TD.

ETTh1 n'a pas d'anomalies étiquetées : le TD en **injecte** quatre types, 12 par période, chacune sur une mesure tirée au hasard.



Fenêtres de **48 heures** : une fenêtre est anormale si elle contient au moins une heure d'anomalie. Au test : 60 fenêtres anormales sur 237.

Source : ETTh1, période de validation, anomalies injectées par la fonction du TD (graine 1). Calculé pour le cours.

L'autoencodeur : compresser puis reconstruire

Pourquoi : le modèle de l'étape 5.

L'**encodeur** résume la fenêtre en un petit code ; le **décodeur** doit la reconstruire à partir de ce résumé. Trop petit pour tout recopier, le code ne garde que **ce qui revient souvent**.



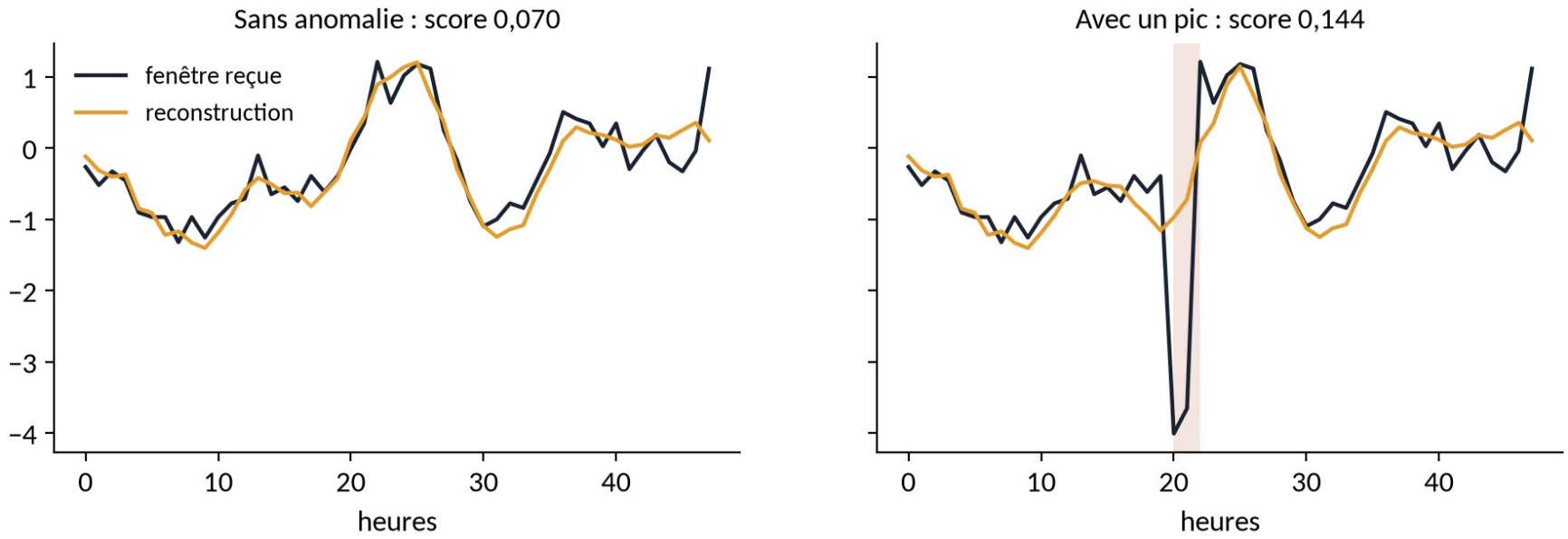
Entraîné sur **2 149 fenêtres sans anomalie** (une toutes les 4 heures), pour que la reconstruction ressemble à la fenêtre. 7 407 paramètres.

Source : Goodfellow, Bengio et Courville, Deep Learning, chapitre 14 ; paramètres comptés sur le modèle du TD.

L'erreur de reconstruction comme score

Pourquoi : transformer une reconstruction en score d'anomalie : le TODO 7.

Le **score** d'une fenêtre : l'écart moyen, au carré, entre la fenêtre et sa reconstruction. Plus il est haut, plus la fenêtre est **suspecte**.



La même fenêtre de validation (mesure HULL), sans puis avec un pic injecté.

× 2

le score double avec le pic : de 0,070 à 0,144

1 sur 10

fenêtre sans anomalie dépasse déjà 0,174 (validation)

Le score **classe** les fenêtres ; décider d'alerter demande un **seuil** (partie 8). Au TODO 7 : une erreur quadratique moyenne par fenêtre.

Source : ETTh1, période de validation, autoencodeur du TD entraîné 30 époques. Calculé pour le cours.

Les détecteurs classiques : ACP et Isolation Forest

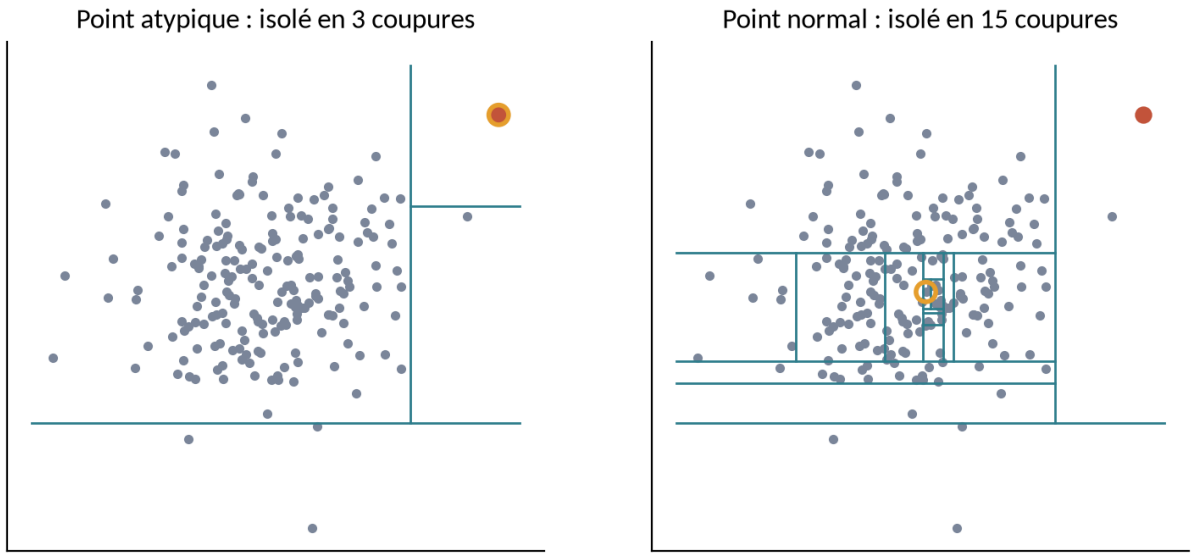
Pourquoi : les deux baselines de la détection d'anomalies.

Deux baselines classiques, sur les mêmes fenêtres : l'**ACP** reconstruit avec une compression **linéaire** ; l'**Isolation Forest** compte les **coupures au hasard** qu'il faut pour isoler un point.

ACP

- Garde les directions qui expliquent **90 %** de la variance
- Reconstruit la fenêtre à partir d'elles
- Score : l'**erreur de reconstruction**, comme l'autoencodeur

Isolation Forest



La figure montre un tirage. Moyenne sur 200 tirages : **4,0** coupures pour le point atypique, **12,9** pour le point normal le plus central.

Source : Liu, Ting et Zhou, Isolation Forest, ICDM 2008 ; scikit-learn, Novelty and Outlier Detection. Exemple jouet calculé pour le cours.

L'autoencodeur en PyTorch

Pourquoi : les formes des tenseurs, et le score en une ligne.

```python
import torch
import torch.nn as nn

encodeur = nn.Sequential(
    nn.Conv1d(7, 16, 5, stride=2, padding=2), nn.ReLU(),     # 48 -> 24
    nn.Conv1d(16, 32, 5, stride=2, padding=2), nn.ReLU(),    # 24 -> 12
    nn.Conv1d(32, 8, 3, padding=1))                          # 8 canaux
decodeur = nn.Sequential(
    nn.ConvTranspose1d(8, 32, 4, stride=2, padding=1), nn.ReLU(),
    nn.ConvTranspose1d(32, 16, 4, stride=2, padding=1), nn.ReLU(),
    nn.Conv1d(16, 7, 3, padding=1))                          # 12 -> 48

x = torch.randn(5, 48, 7)        # 5 fenêtres, 48 heures, 7 capteurs
code = encodeur(x.transpose(1, 2))
print(code.shape)                # torch.Size([5, 8, 12]) : 96 nombres
reconstruction = decodeur(code).transpose(1, 2)
print(reconstruction.shape)      # torch.Size([5, 48, 7])

score = ((reconstruction - x) ** 2).mean(dim=(1, 2))
print(score.shape)               # torch.Size([5]) : un score par fenêtre
```

- **Conv1d** de pas 2 : la longueur est divisée par 2, deux fois
- **ConvTranspose1d** de pas 2 : elle est multipliée par 2
- Le code : **8 × 12 = 96** nombres, contre 48 × 7 = 336
- Le score : une **erreur quadratique moyenne** par fenêtre (TODO 7)

Source : Documentation PyTorch : torch.nn.Conv1d, torch.nn.ConvTranspose1d.

L'autoencodeur : à retenir

1. On n'apprend que **le normal** ; une anomalie est ce que le modèle n'explique pas.

2. L'**autoencodeur** compresse la fenêtre (336 nombres en 96) puis la reconstruit ; le **score** est l'écart quadratique moyen entre la fenêtre et sa reconstruction.

3. Deux baselines : l'**ACP**, même idée avec une compression linéaire, et l'**Isolation Forest**, qui isole les points atypiques par des coupures au hasard.

Pour approfondir : Goodfellow, Bengio et Courville, Deep Learning, chapitre 14 : Autoencoders ; scikit-learn : Novelty and Outlier Detection



Partie 8

Choisir un seuil d'alerte

Un score ne déclenche rien tant qu'on n'a pas fixé le seuil, et ce choix est une décision métier

Le seuil d'alerte : définition

Définition

Un détecteur donne un score ; l'alerte part quand le score dépasse un seuil. Choisir ce seuil, c'est arbitrer entre fausses alertes et anomalies manquées : on le fixe sur la validation, d'après le coût de chaque erreur.

Le problème que cela résout

Un seuil bas dérange l'opérateur pour rien ; un seuil haut laisse passer des pannes. Aucun calcul ne tranche sans savoir ce que coûte chaque erreur.

Ce qui entre, ce qui sort

Entrée : les scores de la validation, leurs étiquettes et le coût de chaque erreur.
Sortie : un seuil, appliqué tel quel au test.

Ce que l'on va voir, dans l'ordre

2. Précision, rappel et AP
3. Le coût des erreurs
4. Un seuil fixé sur le passé
5. À retenir

Dans le TD : Étape 5 : TODO 8 (précision et rappel) ; étape 6 : le seuil et le coût.

Précision, rappel et AP

Pourquoi : les mesures du Jour 2, appliquées aux alertes : le TODO 8.

Les mesures du Jour 2, appliquées aux alertes : la **précision** pour les fausses alertes, le **rappel** pour les anomalies manquées.

Exemple : 10 alertes, dont 6 vraies anomalies ; 8 anomalies en tout.

Précision

6 / 10 = 60 %

des alertes sont de vraies anomalies

Rappel

6 / 8 = 75 %

des anomalies ont donné une alerte

AP

l'aire sous la courbe précision-rappel, sur tous les seuils. 1 est parfait ; le hasard obtient la part d'anomalies : 0,25 au test.

Une alerte part quand le score est **strictement supérieur** au seuil. C'est la convention du TODO 8.

Source : scikit-learn, section 3.4 : precision_recall_curve, average_precision_score.

Le coût des erreurs

Pourquoi : le seuil d'alerte est une décision métier, celle de l'étape 6.

Chaque erreur a un prix. On choisit le seuil qui rend le **coût total** le plus bas, **sur la validation**.

$$\text{coût total} = 5 \times \text{fausses alertes} + 100 \times \text{anomalies manquées}$$

5 : le déplacement d'un technicien ; 100 : une panne. Ce sont les valeurs par défaut du TD, réglables par deux curseurs.

Seuil bas

30 fausses alertes, 2 oublis

$$5 \times 30 + 100 \times 2 = 350$$

Seuil haut

3 fausses alertes, 6 oublis

$$5 \times 3 + 100 \times 6 = 615$$

Changez les prix, et le meilleur seuil peut changer : le seuil est une **décision métier**, pas une propriété du modèle.

Un seuil fixé sur le passé

Pourquoi : un seuil choisi aujourd'hui sert pendant des mois.

Le seuil se fixe sur **le passé** (la validation), puis s'applique **tel quel** à l'avenir. Cela suppose que les scores **se répartissent de la même façon**.



La série dérive (partie 6) : les scores peuvent dériver aussi. Le TD vous fait comparer les scores de la validation et du test. Le suivi en service est au programme du Jour 6.

Le seuil d'alerte : à retenir

1. **Précision** : la part des alertes qui sont de vraies anomalies. **Rappel** : la part des anomalies qui ont donné une alerte. L'**AP** résume tous les seuils.

2. Le seuil se choisit **sur la validation**, là où le **coût total** est le plus bas : fausses alertes et oublis, chacun multiplié par son coût.

3. Un seuil fixé sur le passé suppose que les scores **se répartissent de la même façon** ensuite : à vérifier, puis à surveiller en service.

Pour approfondir : scikit-learn, section 3.4 : Precision, recall and F-measures

TD · étape 9

La brique projet : la pompe SKAB

Pourquoi : le volet maintenance du projet, à l'étape 9.

Les capteurs d'une **pompe à eau** de banc d'essai, une mesure par seconde. Cette fois, les anomalies sont **réelles** : provoquées, par exemple en fermant une vanne, et étiquetées.

- **8 capteurs** : vibrations, courant, pression, températures, tension, débit
- **16 expériences** du groupe valve1 ; fenêtres de 32 secondes
- Les mêmes détecteurs : autoencodeur, ACP, Isolation Forest

Deux façons de définir le normal

Protocole A	un enregistrement séparé, sans aucune anomalie
Protocole B	les 400 premières secondes de chaque expérience, avant l'anomalie ; test sur la suite



Lequel définit le mieux le normal, et pourquoi ? C'est la question de l'**étape 9**, point de départ du **volet maintenance** du projet (au choix).

Source : Katser et Kozitsin, Skoltech Anomaly Benchmark (SKAB), 2020.

Synthèse

Pourquoi : les réponses aux trois questions, à garder en tête pour le TD.

1. **Préparer sans tricher** : découper dans l'ordre du temps, normaliser avec les statistiques de l'entraînement, couper en fenêtres (96 heures pour en prévoir 24).

2. **Prévoir** : trois baselines (naïve, saisonnière, linéaire) face à trois réseaux, le LSTM (pas à pas), le TCN (convolutions causales dilatées) et PatchTST (attention entre patches) ; normaliser chaque fenêtre contre la dérive.

3. **Détecter et alerter** : apprendre le normal ; score = erreur de reconstruction de l'autoencodeur, face à l'ACP et à l'Isolation Forest ; seuil choisi sur la validation selon le coût des erreurs.

4. **Mesurer avant de conclure** : même protocole pour tous, test une seule fois. Le deep learning ne gagne pas toujours ; c'est le tableau qui tranche.

Déroulé du TD 3

Pourquoi : chaque étape du TD applique une partie du cours.

Étape 2

Explorer ETTh1, découper, normaliser, fenêtres

partie 1 · TODO 1 et 2

Étape 3

Baselines : naïve, saisonnière, régression linéaire

partie 2 · TODO 3

Étape 4

LSTM, TCN et PatchTST, sans puis avec normalisation par fenêtre

parties 3 à 6 · TODO 4 à 6

Étape 5

Autoencodeur face à l'ACP et à l'Isolation Forest

parties 7 et 8 · TODO 7 et 8

Étape 6

Seuil d'alerte et coût métier

partie 8

Étape 7

Fiche de décision ingénieur

synthèse

Étape 8

Bilan : deep learning ou classique ?

TD 1 à 3

Étape 9

Brique projet : la pompe SKAB (volet au choix)

parties 7 et 8

Étapes 0 et 1 : vérifier l'installation et revoir le cours en bref. Tout le TD tourne sur CPU : les modèles sont petits.

Pendant le TD : sur les diapositives du cours, la pastille orange « TD · étape N » en haut à gauche indique l'étape, et le TODO, où elles servent.